

Neural Network-Based Undersampling Techniques

Md Adnan Arefeen¹, Sumaiya Tabassum Nimi², and M. Sohel Rahman³, *Senior Member, IEEE*

Abstract—Machine learning models have gained popularity nowadays for their potential to solve real-life issues when trained on pertinent data. In many cases, the real-life data are class imbalanced and hence the corresponding machine learning models trained on the data tend to perform poorly on metrics like precision, recall, AUC, F1, and G-mean score. Since class imbalance issue poses serious challenges to the performance of trained models, a multitude of research works have addressed this issue. Two common data-based sampling techniques have mostly been proposed—undersampling the data of the majority class and oversampling the data of the minority class. In this article, we focus on the former approach. We propose two novel algorithms that employ neural network-based approaches to remove majority samples that are found to reside in the vicinity of the minority samples, thereby undersampling the former to remove (or alleviate) the imbalance issue. We delineate the proposed algorithms and then test the proposed algorithms on some publicly available imbalanced datasets. We then compare the performance of our proposed algorithms to other popular undersampling algorithms. Finally, we conclude that our proposed algorithms outperform most of the existing undersampling approaches on most performance metrics.

Index Terms—Autoencoder, class imbalance, classification, neural network, undersampling.

I. INTRODUCTION

NOWADAYS, with the advance of machine learning and data mining, many general but significant tasks are carried out by automated machines previously done by human. Classification is one of the main tasks in data mining. Sometimes we face difficulties in doing classification tasks. Class imbalance plays a crucial role behind this difficulty [1]. In a dataset having class imbalance, one class consists of fewer instances than the other class. Due to the lack of instances in the minority class, it is difficult to train them. Class imbalance problem is found as a significant problem in most of the real-world applications, such as intrusion detection [2], fraud detection [3], [4], customer churn prediction [5], rare disease

diagnosis, and text classification [6]. In most cases, correctly classifying the minority class instances is of great interest to the researchers. In fraud detection case, identifying the two or three fraud transaction out of hundreds of real transaction is very important. It is also true for rare disease diagnosis such as cancer. To train the samples belonging to the minority class, several methods have been discussed in the literature. One of the most common methods in this regard is resampling. We can increase the number of minority samples by generating similar data points (known as oversampling). We can also decrease the number of majority samples to keep the dataset more or less balanced (known as undersampling). Some oversampling algorithms, include SMOTE [7], ADASYN [8], MWMOTE [9], and variants of SMOTE [10], [11] are popular in literature. Edited nearest neighbor [12], neighborhood cleaning rule [13], Tomek link [14] are some recognized undersampling algorithms found in literature. In recent years, cost-sensitive learning has been proposed in the literature to solve the class imbalance problem.

This article proposes two neural network-based undersampling algorithms (Section III). We have experimented on eleven real world and publicly available datasets to examine the effectiveness and efficacy of our results (Section IV).

II. LITERATURE REVIEW

Several approaches exist in the literature to deal with the class imbalance issue. Some research works focus on learning suitable distance metrics that can aid in the task of learning from imbalanced dataset [15]. On the contrary, some other researches focus on construction algorithm-based methods to improve the rule-based interpretability of imbalanced data [16]. Another popular approach is to use bagging and boosting techniques for better classification performance on imbalanced dataset [17]–[19]. However, one of the most common techniques to tackle the issue of class imbalance is resampling the dataset to bring the expected balance in the original imbalanced dataset. Most of the resampling algorithms were proposed for the most common case of class imbalance, imbalance found in a two-class dataset. In the case of a two-class dataset, the class in which most data belong is called the majority class and the corresponding samples are called the majority samples. Vice versa, we have minority class and minority samples. In this section, some of the popular resampling algorithms will be reviewed.

A. Oversampling

As discussed previously, oversampling adds minority samples to the original dataset. This addition of minority samples can be done both by using existing minority samples or

Manuscript received November 9, 2019; revised January 7, 2020 and June 13, 2020; accepted July 27, 2020. This article was recommended by Associate Editor Y.-P. Huang. (Corresponding author: Md Adnan Arefeen.)

Md Adnan Arefeen and Sumaiya Tabassum Nimi are with the Department of Computer Science Electrical Engineering, University of Missouri–Kansas City, Kansas City, MO 64110 USA, and also with the Department of Computer Science and Engineering, United International University, Dhaka 1209, Bangladesh (e-mail: adnanarefeen@gmail.com; sumtamnimi@gmail.com).

M. Sohel Rahman is with the Department of Computer Science and Engineering, Bangladesh University of Engineering and Technology, Dhaka 1205, Bangladesh (e-mail: sohel.kcl@gmail.com).

This article has supplementary downloadable material available at <http://ieeexplore.ieee.org>, provided by the authors.

Color versions of one or more of the figures in this article are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TSMC.2020.3016283

by synthesizing new samples. The first technique, called random oversampling, has been explored in [20] and [21]. However, this approach suffers from the issue of creating specific rules, leading to overfitting. The second technique we stated, popularly called synthetic oversampling, may lead to better performance by adding additional information to the dataset. However, much depend on the relevance of the generated samples. Several studies have been conducted focusing on synthetic oversampling, like, SMOTE [7], Borderline-SMOTE [10], ADASYN [8], and GASMOTE [22].

SMOTE [7] makes an effort to generate new (artificial) minority data samples along the line connecting existing minority samples to their nearest neighbors. Whereas in Borderline-SMOTE [10], those minority samples that lie in the borderline of majority and minority samples (hence are more prone to misclassification) are identified and chosen for generating synthetic samples. Another popular oversampling technique is ADASYN [8] that leverages a weight assigned to each minority sample based on the degree of complication to learn the sample. Briefly, this technique generate more synthetic samples for the minority samples that are more complicated to learn and vice versa.

GASMOTE [22] is a genetic algorithm-based oversampling technique. Its actually a variant of the popular SMOTE [7] algorithm. In SMOTE and all other variants of SMOTE, the same sampling rate is assigned to all the minority samples while generating synthetic samples. On the other hand, in GASMOTE, different sampling rates are assigned to each sample in the minority data and a genetic algorithm is employed to find the optimal sampling rates. Finally, it generates a new dataset through oversampling using the optimized sampling rates.

B. Undersampling

As discussed previously, undersampling methods tackle the issue of class imbalance by pruning majority class samples. Several popular undersampling techniques include random undersampling, edited nearest neighbor rule (ENN) [12], Tomek links [14], Near miss (NM) family of algorithms (NM-1, NM-2, NM-3) [23], evolutionary undersampling, etc. Random undersampling can be treated as the most basic approach that removes some majority data samples at random to make the dataset balanced. But this random removal of data points may lead to undesirable information loss from the majority class. Another approach for undersampling, called ENN, was proposed in [12] and is based on k -nearest neighbors. In ENN, those majority data points are pruned whose class label mismatches with the majority of its k -nearest neighbors. Tomek links removal rule (TLL) [14] is another k -nearest neighbors-based method for undersampling. In the Tomek link removal algorithm, pairs of data points called Tomek Links are identified. Tomek links are those pairs of data points that have different class labels despite being each other's nearest neighbors. TLL algorithm removes either the totem links or the majority components thereof. NM family of methods are yet other k -nearest neighbor-based techniques. NM [23] works by removing majority data samples based on their distances from

each other. Although k -nearest neighbor-based techniques for undersampling are most common, these are not the only players in the field. Notably, clustered centroids [24] is another undersampling technique that leverages k -means clustering. Very recently, radial basis function has been used [25] to measure the degree of the characteristic of a given point as a majority class and minority class. The author defined this term as the mutual class potential.

Evolutionary algorithm-based methods for undersampling have also been studied [26]. Moreover, majority data points are chosen based on the assumption by Hou *et al.* [27] that the data points belonging to a class reside closer to each other in the feature space unless the data points are noisy. Hence, noisy data points have relatively lower density compared to the other examples belonging to a class. That is why, to prevent information loss and at the same time, to eliminate the noise in the data, local density peaks were found out by using a new metric to evaluate the approximate probabilistic distribution of majority class samples by kernel density estimation, noisy data points being removed at the same time automatically. Eventually, the local density peaks were merged with the minority data, resulting in the ultimate balanced dataset. Liu *et al.* [28] proposed a novel method, namely, GU-MOACOFS that incorporates ensemble learning, evolutionary under-sampling, and multiobjective feature selection for augmenting the performance of classification algorithms on class imbalanced data. The proposed method comprises of several consecutive steps. At first, Bootstrap is employed to divide the data into some subsets based on the Bagging framework. Then these subsets were undersampled to engender sampling subsets by genetic algorithm based on a proposed metric called V-statistic. Finally, F1 and Gmean scores are used as objective functions of multiobjective ant colony optimization for selection of feature subset. Hence, GU-MOACOFS has the potential to improve the performance on datasets that suffer from the dual problem of class imbalance, as well as high dimensionality. However, a novel technique based on filtering of noises has been proposed by Chen *et al.* [29], for under-sampling by using iterative-partitioning filter (IPF) in conjunction with easy-ensemble (EE) that is capable of getting rid of noises from both majority and minority classes in a binary classification problem. In between under-sampling and Adaboost in EE, the step for IPF is placed. EE returns a number of subsets of training data after undersampling. IPF acts on these subsets, thereby removing noisy examples from both majority and minority classes. Ng *et al.* [30] has introduced a sensitivity-based undersampling method that takes into consideration the distribution of the original train dataset. The proposed method at first divides the majority data into some clusters. Then samples are selected from both clusters of majority class and minority class by means of a stochastic sensitivity-based measure. Repeated clustering and sampling ensures that a balanced dataset is ultimately generated having high cluster sensitivity.

III. METHODS

In this section, we will discuss two major problems in class imbalance problem for binary class data, along with proposing

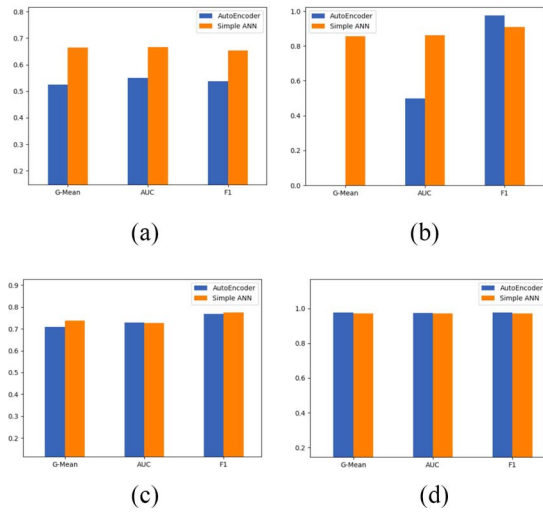


Fig. 1. Selection of threshold value for choosing either an autoencoder or a simple neural network. G-mean, AUC, and F1 performance metric values are comparatively higher if the data having less than 30 features is trained using a simple neural network than an autoencoder and vice versa. Notice that, G-mean for Pima dataset is 0 for autoencoder since it fails to classify one class entirely. (a) Haberman. (b) Pima. (c) wpbc. (d) wdbc.

algorithms for solving the problem of class imbalance through undersampling and also discuss how the proposed two methodologies solve those problems. The first drawback of any imbalanced data is the existence of a nearly overlapped region of two different classes. It is very hard for a model to learn the minority data in such an environment. The second problem is the existence of a high imbalance ratio between the two classes of data. The imbalance ratio is defined as follows:

$$\text{Imbalance Ratio} = \frac{M^+}{M^-}.$$

Here, M^+ refers to the number of majority samples in the data. M^- refers to the number of minority samples. The higher the ratio, the more difficult it is for a model to learn the minority samples. We propose two algorithms NUS-1 and NUS-2 to solve the stated problems. NUS-1 is a hard undersampling algorithm using the neural network. NUS-2 is a soft undersampling using neural network. NUS-2 is an extended and more flexible version of the NUS-1 algorithm.

Depending on a threshold, we either use a simple artificial neural network (simple ANN) or a sophisticated autoencoder to train the minority class. Fig. 2(a) and (b) from the supplementary file depict two such models. We empirically observe that encoding of an input having a few features using an autoencoder (having less neurons than input size in a hidden layer) has a drastic effect on the performance of the model [Fig. 1(b)]. More specifically, for a dataset having more than 30 attributes, we use an autoencoder; otherwise, a simple ANN is exploited for training. This simple ANN uses 3 hidden layers with the number of neurons greater than the number of features (10 neurons in each layer if the number of input features are less than 10; 50 neurons in each layer for dataset having the number of features 10 to 30) as we do not see over-fitting as a major concern in our context. And understandably so, since we have to generate a minority sample with as high accuracy

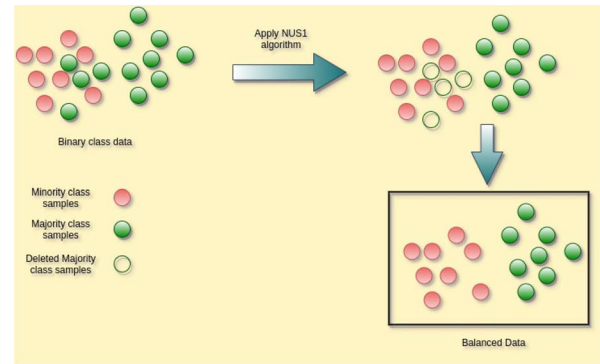


Fig. 2. Pictorial representation of NUS-1 algorithm.

as possible (i.e., approximately 100% accuracy). If we fail to fit the minority samples well enough, we are bound to lose information on predicting majority samples. If the model is not strong enough, it may propagate error when predicting majority samples. We also followed the same procedure at the time of training with an autoencoder for the dataset having the number of features greater than 30. The only difference was the training of the data with an autoencoder rather than simple ANN. The first layer consists of the same number of neurons as the number of features. Then the encoded layer consists of half of the neurons of the previous layers. After that, the decoding layers are as same as the encoding layers. The final layer neurons are the same in number as the input layer. However, one can use different suitable threshold values based on the data. Based on the performance metric values, it is observed that if the data has less the 30 features, the performance metric value is higher when trained with simple ANN rather than with an autoencoder and vice versa. Comparisons of AUC, G-Mean, and F1 values are shown in Fig. 1 for different datasets. Descriptions of the datasets are shown in Table II and discussed in the result analysis section. From Fig. 1, significant differences among metric values have been observed for Pima and Haberman datasets. Same observation is also found in the Balance dataset (see Supplementary File Fig. 1). As both the datasets have fewer than 30 features, simple ANN outperforms the autoencoder. For wdbc dataset, the AUC, G-mean, and F1 values are almost similar for both simple ANN and autoencoder as the dataset has 30 features. Same is the case for wpbc dataset having 33 features. So, if the number of features increases, it is suitable to use autoencoder rather than simple ANN. An autoencoder can extract the best features from a large number of features. Having justified the choice of the neural network to train on based on the number of features, we are going to discuss our proposed algorithms for undersampling in the subsequent sections.

A. Undersampling Algorithm

1) *Algorithm 1—Hard Undersampling Using Neural Network (NUS-1)*: The main idea of the NUS-1 algorithm is to remove the majority class data that are more similar to minority class data. It aims to remove the data that are (nearly) overlapped with the minority class data. It chooses the number of majority samples which is exactly equal to the number of

minority samples thereby making the dataset completely balanced (i.e., the ratio of majority samples to minority samples will be 1). Notably, this may result in the removal of a lot of majority data points which may not be desirable in some practical scenarios (please see Section V). The NUS-1 algorithm is pictorially represented in Fig. 2.

Assume that, in our dataset, we have, respectively, n_1 and n_2 minority and majority samples. We train a neural network (a simple ANN or a sophisticated autoencoder based on the already discussed threshold) to learn the minority samples (i.e., the feature values of the n_1 samples). Then, we predict the features of the (n_2) majority samples using that trained model. We calculate the Euclidean distances between the real feature values and the predicted ones (Line 14) and store those in the `distArray` list mapped by the indices of the respective majority class samples. This list is then sorted in descending order based on the Euclidean distances. From this sorted list the first n_1 data samples are chosen (Line 19) and are combined with the n_1 minority class data from the original dataset. This is the final dataset returned by the algorithm. So in effect, the n_1 majority samples chosen by the algorithm are the ones that are the farthest from the predicted values in terms of Euclidean distance, i.e., we are essentially removing the majority class samples that are present in the vicinity of the minority class samples and retaining only those that are located further therefrom. Consequently, the decision boundary becomes more defined making the (resulting balanced) dataset more separable. As will be reported later, this algorithm performs extremely well for most of the datasets considered in this research. We note however that, this algorithm is better-suited when there is no overlap between the data points, as will be evident in Section III-C. In addition, since we have the sorted distance list, we can actually select not only the equal number of minority samples but also select no of majority samples greater or less than the number of minority samples. So, it can be said that the number of chosen majority samples can act as a parameter to the algorithm. But the question is how many samples should we choose to get better results as well as without losing so much information from the majority data. Our second algorithm called NUS-2 will try to answer the question. The NUS-2 algorithm can also be useful in overcoming problems due to nearly overlapping data. The NUS-2 algorithm will be described in detail in the next section.

2) *Algorithm 2—Undersampling Based on Soft Neural Network (NUS-2)*: The hard neural network-based undersampling algorithm described above always selects the equal number of minority data samples based on the distances that are far from the minority samples. However, if the majority and minority data are almost overlapped, things become more complicated. We also do not know how far the minority data is from each other. We propose a new algorithm, namely, the soft neural network-based undersampling to resolve this issue. The soft version (NUS-2) differs from the hard version in the selection strategy adopted. Lines 1–9 are identical in both versions. In Lines 12–18, we use the model that was fitted on the samples from the minority class to predict the minority samples. We then sort the list in descending order and get the average of the greater half distances of the minority samples.

Algorithm 1 Hard Under-Sampling Using Neural Network

```

1:  $n_1 \leftarrow$  size of the minority class
2:  $n_2 \leftarrow$  size of the majority class
3:  $m \leftarrow$  number of features/attributes
4:  $majSamples[1 \dots n_2] \leftarrow$  Majority class samples
5:  $minSamples[1 \dots n_1] \leftarrow$  Minority class samples
6: if  $m > threshold$  then
7:    $model \leftarrow autoencoder.train(minSamples[1 \dots n_1])$ 
8: else
9:    $model \leftarrow simpleANN.train(minSamples[1 \dots n_1])$ 
10: end if
11:  $distArray \leftarrow \{\}$ 
12: for each  $x \in n_2$  do
13:    $x' \leftarrow model.predict(x)$ 
14:    $d \leftarrow ||x - x'||_2^2$ 
15:    $index \leftarrow x.index$ 
16:    $distArray \leftarrow distArray \cup \{d, index\}$ 
17: end for
18:  $sortedList \leftarrow sort(majSamples[1 \dots n_2], distArray)$   $\triangleright$  Sort the indices of  $n_2$ 
   samples according to descending order of distance
19:  $selectedIndices \leftarrow sortedList[1..n_1]$   $\triangleright$  select first  $n_1$  number of indices from the
    $sortedList$ 
20:  $X_1 = \{\}$ 
21: for each  $index \in selectedIndices$  do
22:    $X_1 = X_1 \cup majSamples[index]$ 
23: end for
24:  $finalData = X_1 \cup minSamples[1 \dots n_1]$ 

```

In Line 19, we save the distance values in $lastMid_{avg}$. After that, we use the same model to predict the samples of the majority class. This time we adopt the following approach to choose the samples of the majority class. We generate the clone of a sample by feeding it to the model, and calculate the Euclidean distance between the actual and generated sample. If the distance is higher than $lastMid_{avg}$, we include it in the final dataset as a majority class data point. The soft neural network-based undersampling algorithm performs very well for sampling overlapping data, as will be observed in the next section. Notably, apart from its distinctive quality to handle overlapped data points, NUS-2 also differs with NUS-1 in that it does not aim to make the majority-minority samples ratio 1, thereby making it a better alternative in cases where we have only a handful of minority samples.

B. Comparison Between the Proposed Algorithms

It is observed from the classification results and Figs. 3 and 4 that NUS-1 performs well in a dataset where data points exhibit less overlapping. NUS-1 algorithm actually retains those data points from the majority class which are the most distant from most of the data points belonging to the minority class. But in case of a high degree of overlapping, there could be some minority samples overlapped with the retained majority samples. Clearly, this issue is (almost) nonexistent when the degree of overlapping is minimal. Hence, NUS-1 makes a dataset balanced as well as linearly separable. On the contrary, NUS-2 first calculates the average distance of minority samples to its generator samples generated by the model and uses that to find the perimeter of minority samples. Then it retains those majority samples that are outside of the computed perimeter. Thus, in the sequel, the overlapping is removed. Hence, NUS-2 performs better when the dataset exhibits a high degree of overlapping. However, the choice of distance is a tunable parameter. Interestingly, NUS-1 and NUS-2 provide us with an approach to indirectly verify the

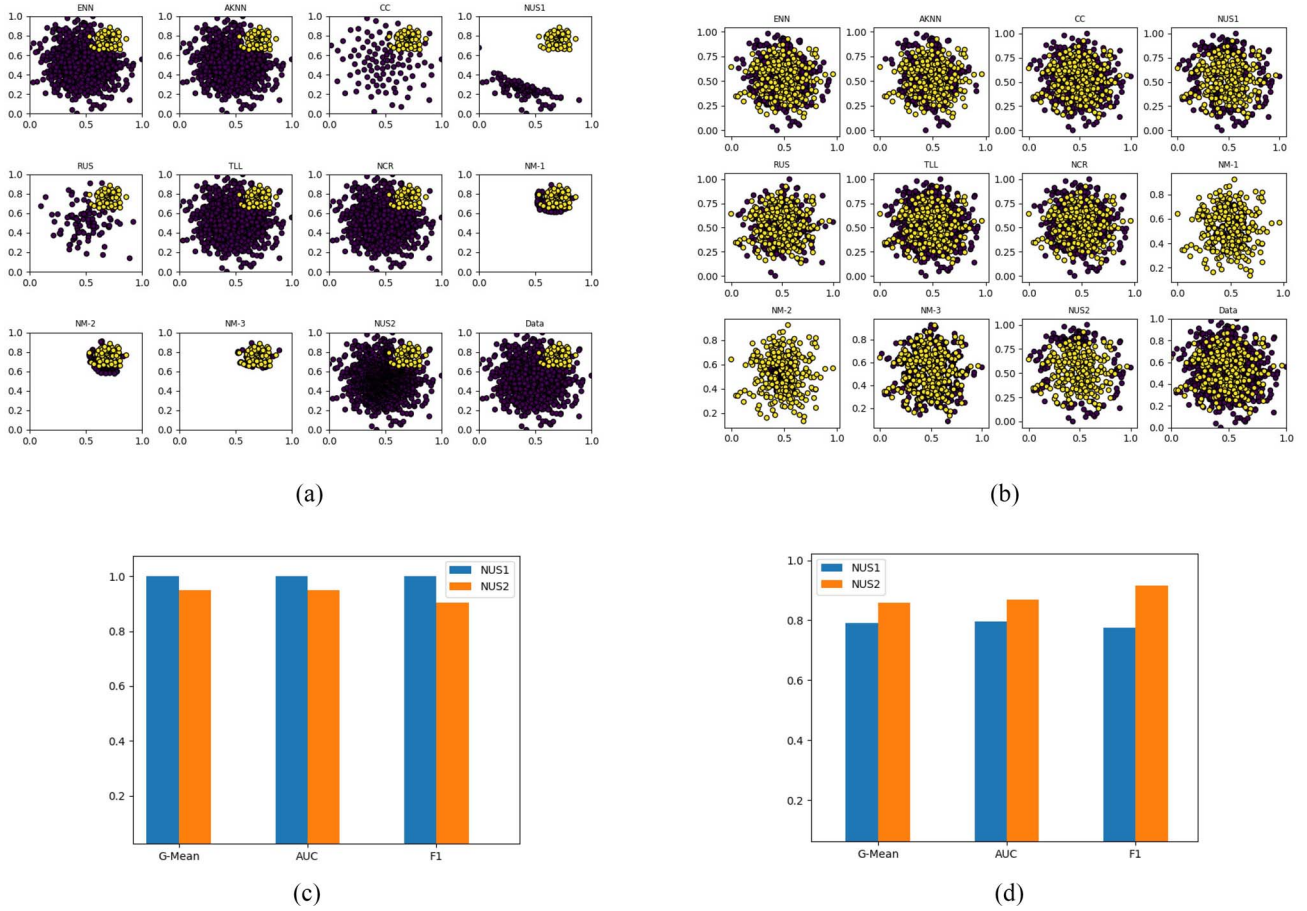


Fig. 3. Effect of undersampling algorithms on two different datasets. (a) NUS-2 selects 977 majority data points out of 1000 data where NUS-1 selects 100 majority data points, equal to the number of minority data points. Undersampling by other algorithms is also shown in the figure. (b) NUS-2 selects only 126 majority data points out of 1000 majority data where the number of minority data points is 300. For a nearly overlapping dataset, NUS-2 removes majority data points that are similar to minority points based on a distance value. Pictorial view of selection of data points by other undersamplers are also shown for comparison. (c) NUS-1 algorithm got 100% G-Mean, AUC, and F1 score. NUS-2 has higher than 90% of these metric values. (d) NUS-2 gets higher performance metric values than NUS-1. In comparison with NUS-1, NUS-2 is almost 6% higher in G-Mean, 7% higher in AUC score and 14% higher in F1 score.

nature of the dataset (i.e., whether the dataset has a high degree of overlapping or not).

C. Undersampling on Artificial Dataset

In this section, we analyze the effect of NUS-1 and NUS-2 as undersamplers through visualization. We have used the scikit-learn package [31] and experimented first on two artificial datasets. The first dataset comprises 1000 majority samples and 100 minority samples (i.e., the majority to minority samples ratio is 10:1). The dataset has two features, which makes it easy to plot the dimensions and visualize the data. The centers of the two clusters are $[0.0, 0.0]$ and $[2.0, 2.0]$, respectively. The standard deviations of the cluster samples from the centers of the respective clusters are 1.5 and 0.5, respectively. The effect of NUS-1 and NUS-2 is illustrated in Fig. 3(a).

Next, we generate the second dataset with a high degree of overlapping, i.e., the majority and minority samples are overlapped with each other. In this dataset, the majority to minority ratio is 3.33:1 with 1000 majority samples (i.e., same as the first dataset) and 300 minority samples. We choose the center of the two classes to be $[0.0, 0.0]$ and $[0.02, 0.05]$,

respectively, which makes the data points of the two classes overlapped with each other. The standard deviations of the two cluster samples from the center are $[1.5, 1.5]$, respectively. The pictorial representation of data chosen by each sampler is shown in Fig. 3(b). In Fig. 3(a), NUS-1 selects the farthest 100 samples out of 1000 samples. In this case, NUS-2 selects 977 majority samples. For the second dataset, as the majority class data is nearly overlapping with the minority class data, NUS-2 selects only 126 majority data points. As NUS-2 considers distances of minority class, it has to remove the majority data points whose distances are not higher than the $lastMid_{avg}$ as discussed earlier. As a result, for the second case, NUS-2 works better than NUS-1. It is to be noted that the $lastMid_{avg}$ can be an input parameter of the NUS-2 algorithm. One can replace the $lastMid_{avg}$ value by some other conditions to get the required number of majority data points. The performance metric values G-mean, AUC, and F1 are reported in Fig. 3(c) and (d).

Now we observe and discuss a particular case where 1) the majority class data consists of outliers, i.e., data points that are far away from the minority data points and 2) the ratio of majority to minority is very high. This situation may arise

Algorithm 2 Soft Under-Sampling Using Neural Network

```

1:  $n_1 \leftarrow$  Size of the minority class
2:  $n_2 \leftarrow$  Size of the majority class
3:  $m \leftarrow$  number of features/attributes
4:  $majSamples[1 \dots n_2] \leftarrow$  Majority class samples
5:  $minSamples[1 \dots n_1] \leftarrow$  Minority class samples
6: if  $m > threshold$  then
7:    $model \leftarrow autoencoder.train(minSamples[1 \dots n_1])$ 
8: else
9:    $model \leftarrow simpleANN.train(minSamples[1 \dots n_1])$ 
10: end if
11:  $minDistArray \leftarrow \{\}$ 
12: for each  $x \in n_1$  do
13:    $x' \leftarrow model.predict(x)$ 
14:    $d \leftarrow ||x - x'||_2^2$ 
15:    $index \leftarrow x.index$ 
16:    $minDistArray \leftarrow minDistArray \cup \{d, index\}$ 
17: end for
18:  $sortedMinorityIndexList \leftarrow$  Sort (in descending order) the indices of  $n_1$  samples based on distance in  $minDistArray$ 
19:  $lastMid_{avg} \leftarrow$  Average distance of half of the minority samples whose indices are in first half of  $sortedMinorityIndexList$ 
20:  $majDistArray \leftarrow \{\}$ 
21: for each  $x \in n_2$  do
22:    $x' \leftarrow model.predict(x)$ 
23:    $d \leftarrow ||x - x'||_2^2$ 
24:    $index \leftarrow x.index$ 
25:    $majDistArray \leftarrow majDistArray \cup \{d, index\}$ 
26: end for
27:  $sortedMajorityList \leftarrow$  Sort in descending order the indices of  $n_2$  samples based on distance of  $majDistArray$ 
28:  $selectedIndices \leftarrow \{\}$ 
29: for each index in  $sortedMajorityList$  do
30:    $dist \leftarrow MajoritySamples[index].dist$ 
31:   if  $dist > lastMid_{avg}$  then
32:      $selectedIndices \leftarrow selectedIndices \cup index$ 
33:   end if
34: end for
35:  $X_1 \leftarrow \{\}$ 
36: for each index in  $selectedIndices$  do
37:    $X_1 \leftarrow X_1 \cup majSamples[index]$ 
38: end for
39:  $finalData \leftarrow X_1 \cup minSamples[1 \dots n_1]$ 

```

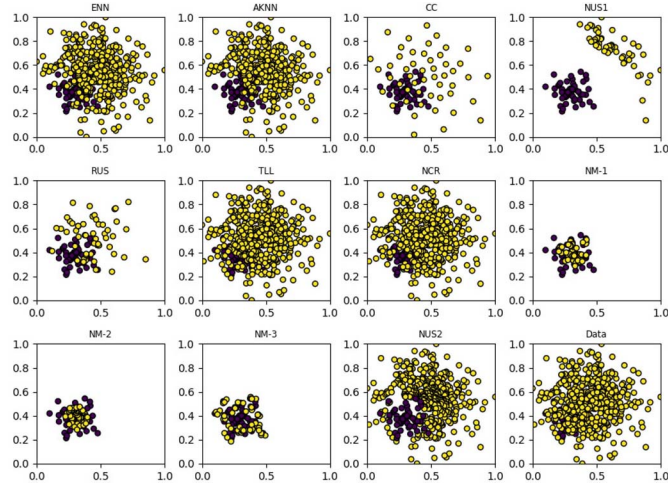


Fig. 4. Comparison of different undersampling algorithms in two datasets having different center and variance. NUS-1 and NUS-2 both are trying to choose far samples from minority class. In the figure, the coordinates of the centers are chosen as $[0.0, 0.0]$, $[5.0, 5.0]$, respectively. The standard deviations from center are 2.5 for the first and 5.5 for the later one. We have used `make_blob` function from scikit-learn [31] to generate the data points. The center and standard deviation are chosen in a way that we can produce some outlier data points in majority class.

due to a certain distribution of the data. In such cases, before applying the proposed undersampling algorithms, the above-mentioned outliers should be removed. We have generated another artificial dataset using scikit-learn [31] package where

TABLE I
UNDERSAMPLING AND CLASSIFIER ALGORITHMS

Undersampling Algorithms	Edited Nearest Neighbour (ENN) [12] All KNN (AKNN) [14] Near Miss (NM-1 NM-2 NM-3) [23] Neighbourhood Cleaning Rule (NCR) [13] Random Undersampling (RUS) Tomek Link (TLL) [14] Radial-Based Undersampling (RBU) [25]
Classifiers	Gradient-boosting (GradBoost) [32] Stochastic gradient descent (SGD) [33] Random forest (RF) [34] K-nearest neighbour [35] Logistic Regression (LR) [36]

the majority to minority ratio is 500:50 (see Fig. 4 for a visualization). When applied on this dataset, NUS-1 and NUS-2 always select the 50 points that are located far from the minority class. In the case of outliers, it may happen that the algorithms always choose the outlier data points. The dataset and the effect of undersampling using NUS-1 and NUS-2 thereon can be visualized in Fig. 4.

IV. RESULTS

A. Overview of the Experiments

We first undersample the datasets under consideration using NUS-1, NUS-2, and other state-of-the-art undersampling algorithms. Subsequently, we apply various classifier algorithms on the undersampled datasets and evaluate the classification results thereof. The classifiers and undersampling algorithms considered in our experiments are listed in Table I.

We have used scikit-learn, scipy, numpy, pandas packages to implement all these algorithms listed in Table I and for data conversion [24], [31], [37], [38]. Keras package has been used to implement the simple ANN and the autoencoder [39]. For graphical representation, we have used the matplotlib package [38].

Once a dataset has been undersampled using one of the undersamplers listed in Table I, we apply one of the classifier algorithms from the same table on the dataset. We have used repeated stratified K-fold to split the undersampled dataset into train and test data.

B. Datasets

We have used 11 real-world datasets to experimentally evaluate our proposed algorithms. All of them are publicly available at UCI machine learning repository [40] and KEEL repository [41] and are benchmark datasets that are used to evaluate the performance of sampling algorithms. The description of the datasets are available in Table II. The *#attribute* indicates the number of features in each dataset. The *#maj* and *#min* refers to the number of majority and minority samples, respectively. The imbalanced ratio, IR is defined as $\#maj/\#min$.

C. Evaluation Criteria

For evaluating the performance of our proposed algorithm, we use some receiver operating characteristics (ROCs)

TABLE II
DESCRIPTION OF DATASETS

Dataset	#attribute	#min	#maj	IR
Ionosphere	34	126	225	1.78
Balance	4	49	576	11.8
Pima	8	268	500	1.9
Satimage	36	626	5809	9.27
wdbc	33	46	102	2.22
wdbc	30	212	357	1.68
haberman	3	81	225	1.77
yeast_4	8	51	1433	28.09
yeast_5	8	163	1321	8.10
yeast_6	8	35	1449	41.4
yeast_05679_4	8	51	477	9.35

curve [42] based performance metrics. The chosen metrics are: area under the ROC curve [35] popularly known as AUC, geometric mean score popularly known as G-mean and F1 measure. The detailed definitions and mathematical formulations of these metrics have been given in the supplementary file (I-B).

D. Results Analysis

We have shown AUC values for some datasets (6 out of 11) having a high imbalance ratio in this article in Table IV. The results of other datasets using all performance metric values are included in the supplementary file [see supplementary Table(s) IV(s)–VI(s) in supplementary file]. We have reported the mean and standard deviation of these metric values produced by each classifier for comparison. The highest and the second-highest values are shown in all tables in boldface.

Notably, the six datasets reported in the main part of this article are: 1) Balance; 2) Satimage; 3) wdbc; 4) yeast_4; 5) yeast_5; and 6) yeast_6. Almost all of these datasets have a high imbalance ratio. The datasets Satimage and wdbc are chosen as both of them have at least 30 attributes. Recall that, 30 is the threshold used to choose the type of neural network in our algorithms as discussed in Section III. Informatively, yeast_4, yeast_5, and yeast_6 have less than 30 attributes.

The number of majority samples selected by each undersampling algorithm are described in Table III of the supplementary file. From Table IV in this article and from supplementary Table(s) IV–VI in supplementary file, it is observed that for a few datasets like Haberman, AKNN works better than NUS-1 and NUS-2. For yeast dataset, NUS-1 and NUS-2 have better performance in metric values than other algorithms. In almost all cases, we notice that, our proposed undersamplers, NUS-1 and NUS-2 outperform all other undersamplers in case of almost all training algorithms especially the results are significant for SGD classifier. NUS-1 and NUS-2 resample the data in such a way that they become more separable as noted from Figs. 3(a) and (b) and 4. This leads to higher AUC, G-mean, and F1 values and hence significantly better performance. We have compared our algorithms with different classifiers so that we can claim that our algorithms are not classifier dependent.

TABLE III
WILCOXON TEST RESULT p VALUES

Metric		vs	ENN	AKNN	NM1	NM2	NM3	CC	NCR	TLL	RUS	
G-Mean	GradBoost	NUS1	0.0232	0.0409	0.0033	0.0051	0.0033	0.0409*	0.0619*	0.0033	0.0033	
		NUS2	0.0076	0.0163	0.0033	0.2132*	0.0033	0.7897*	0.0262	0.0033	0.0164	
	SGD	NUS1	0.0128	0.0208	0.0044	0.0033	0.0033	0.0033	0.0076	0.0033	0.0033	
		NUS2	0.0099	0.0284	0.0058	0.0033	0.0033	0.0208	0.0128	0.0033	0.0262	
	KNN	NUS1	0.0058	0.0329	0.0033	0.0044	0.0033	0.0033	0.0076	0.0033	0.0033	
		NUS2	0.0044	0.0503*	0.0033	0.0044	0.0033	0.0033	0.0044	0.0033	0.0076	
	RF	NUS1	0.0232	0.0409	0.0033	0.0033	0.0033	0.0284	0.0329	0.0033	0.0033	
		NUS2	0.0093	0.0208	0.0033	0.7213*	0.0033	0.7221*	0.0207	0.0033	0.1307*	
	LR	NUS1	0.0128	0.0366	0.0099	0.0033	0.0033	0.0033	0.0208	0.0033	0.0033	
		NUS2	0.0076	0.0262	0.0128	0.0033	0.0033	0.0329	0.0076	0.0033	0.0912*	
	F1	GradBoost	NUS1	0.0329	0.1195*	0.0033	0.0044	0.0033	0.0618*	0.0754*	0.0099	0.0033
			NUS2	0.0208	0.1823*	0.0076	0.6566*	0.0076	0.8589*	0.0367	0.0076	0.091*
SGD		NUS1	0.0099	0.0164	0.0044	0.0033	0.0033	0.0033	0.0128	0.0044	0.0033	
		NUS2	0.0067	0.0128	0.0076	0.0164	0.0058	0.0754*	0.0128	0.0058	0.1549*	
KNN		NUS1	0.0067	0.0164	0.0033	0.0033	0.0033	0.0033	0.0128	0.0033	0.0033	
		NUS2	0.0058	0.1095*	0.0033	0.0112	0.0033	0.0058	0.0076	0.0044	0.0164	
RF		NUS1	0.0329	0.0912*	0.0033	0.0033	0.0033	0.0262	0.0505*	0.0099	0.0033	
		NUS2	0.0099	0.0262	0.0128	0.5337*	0.0076	0.8785*	0.0166	0.0076	0.1549*	
LR		NUS1	0.0262	0.0409	0.0099	0.0033	0.0033	0.0033	0.0262	0.0076	0.0033	
		NUS2	0.0076	0.0262	0.0328	0.0329	0.0262	0.1307*	0.0128	0.0067	0.1549*	
AUC		GradBoost	NUS1	0.0329	0.0409	0.0033	0.0044	0.0033	0.0619	0.0505	0.0033	0.0033
			NUS2	0.0128	0.0682*	0.0033	0.1823*	0.0033	0.8587*	0.0409	0.0033	0.0058
	SGD	NUS1	0.0208	0.0262	0.0076	0.0033	0.0033	0.0033	0.0125	0.0033	0.0033	
		NUS2	0.0099	0.0164	0.0058	0.0033	0.0033	0.0058	0.0058	0.0033	0.0099	
	KNN	NUS1	0.0076	0.0329	0.0033	0.0033	0.0033	0.0033	0.0076	0.0033	0.0033	
		NUS2	0.0044	0.0619*	0.0033	0.0044	0.0033	0.0033	0.0044	0.0033	0.0044	
	RF	NUS1	0.0262	0.0409	0.0033	0.0051	0.0033	0.0262	0.0409	0.0033	0.0051	
		NUS2	0.0076	0.0262	0.0033	0.4446*	0.0033	0.5337*	0.0076	0.0033	0.0619	
	LR	NUS1	0.0409	0.0912*	0.0208	0.0033	0.0033	0.0033	0.0409	0.0033	0.0033	
		NUS2	0.0099	0.0409	0.0128	0.0033	0.0033	0.0076	0.0127	0.0033	0.0505*	

To prove the significance, we perform the Wilcoxon signed-rank test result [43] and calculated the p -value. If $p < 0.05$, then we can conclude that the comparison is significantly different. Supplementary Table II shows the Wilcoxon test results for comparison of AUC values between ENN versus NUS-1 and ENN versus NUS-2. For every dataset, we accumulate the computed AUC values where ENN is done before classification. We follow the same ways to get the computed AUC values for NUS-1 and NUS-2. Then we perform the Wilcoxon signed-rank test result for p -value. The Wilcoxon signed-rank test tests the null hypothesis that whether two paired samples come from the same distribution or not. We observe that for eight datasets out of eleven, NUS-1 is better than ENN. In nine datasets out of eleven, NUS-2 is better than ENN. The p -value 0.033 and 0.013 proves that the mean AUC value of NUS-1 and NUS-2 are significantly different from ENN. In this case, we use the AUC values that are evaluated when the GradBoost classifier is used. The details of p -value after comparing each undersampler with NUS-1 and NUS-2 for every classifier is shown in Table III. It is to be noted that we have used a number of classifiers to verify that the proposed undersampling algorithms are not classifier dependent.

V. DISCUSSION

Since NUS-1 aims to remove the data imbalance completely, it removes a number of majority data points. As an example, for Satimage dataset, with 5809 majority and 626 minority samples, NUS-1 selects only 626 majority samples; and so does several other undersampling approaches, e.g., NM-1, NM-2, NM-3, CC, RUS, RBU as well (supplementary file Table III). NUS-1 performs quite well compared to the other algorithms even after removing so many majority data points [Table IV, Table V (supplementary file), Table VI

TABLE IV
AUC VALUES FOR DIFFERENT DATASETS USING VARIOUS UNDERSAMPLERS AND CLASSIFIERS

Dataset	Method	GradBoost	SGD	KNN	RF	LR
Balance	ENN	0.498 ± 0.060	0.500 ± 0.006	0.554 ± 0.111	0.498 ± 0.030	0.500 ± 0.000
	AKNN	0.511 ± 0.063	0.500 ± 0.000	0.595 ± 0.090	0.516 ± 0.058	0.500 ± 0.000
	NM1	0.417 ± 0.193	0.497 ± 0.145	0.513 ± 0.188	0.278 ± 0.147	0.526 ± 0.179
	NM2	0.783 ± 0.155	0.496 ± 0.155	0.744 ± 0.144	0.772 ± 0.157	0.415 ± 0.190
	NM3	0.447 ± 0.152	0.494 ± 0.129	0.636 ± 0.202	0.366 ± 0.209	0.492 ± 0.175
	NUS1	0.887 ± 0.119	0.860 ± 0.215	0.971 ± 0.086	0.897 ± 0.117	0.897 ± 0.122
	NUS2	0.809 ± 0.148	0.768 ± 0.181	0.842 ± 0.114	0.796 ± 0.128	0.721 ± 0.143
	CC	0.964 ± 0.087	0.476 ± 0.142	0.582 ± 0.162	0.944 ± 0.098	0.359 ± 0.154
	NCR	0.494 ± 0.022	0.501 ± 0.008	0.518 ± 0.066	0.497 ± 0.008	0.500 ± 0.000
	TLL	0.495 ± 0.013	0.500 ± 0.000	0.496 ± 0.031	0.493 ± 0.010	0.500 ± 0.000
	RUS	0.561 ± 0.181	0.519 ± 0.099	0.657 ± 0.201	0.522 ± 0.201	0.487 ± 0.194
	RBU	0.913 ± 0.135	0.909 ± 0.122	0.935 ± 0.111	0.924 ± 0.099	0.910 ± 0.116
Satimage	ENN	0.838 ± 0.032	0.556 ± 0.149	0.895 ± 0.032	0.832 ± 0.036	0.516 ± 0.015
	AKNN	0.852 ± 0.045	0.566 ± 0.171	0.902 ± 0.031	0.852 ± 0.045	0.516 ± 0.013
	NM1	0.823 ± 0.042	0.566 ± 0.111	0.790 ± 0.056	0.834 ± 0.042	0.705 ± 0.058
	NM2	0.831 ± 0.052	0.649 ± 0.192	0.832 ± 0.044	0.851 ± 0.042	0.733 ± 0.043
	NM3	0.692 ± 0.050	0.524 ± 0.061	0.688 ± 0.043	0.715 ± 0.048	0.572 ± 0.056
	NUS1	0.994 ± 0.009	0.993 ± 0.014	0.999 ± 0.004	0.997 ± 0.006	0.976 ± 0.023
	NUS2	0.897 ± 0.036	0.993 ± 0.014	0.920 ± 0.027	0.899 ± 0.035	0.862 ± 0.038
	CC	0.912 ± 0.035	0.545 ± 0.142	0.863 ± 0.046	0.818 ± 0.044	0.793 ± 0.048
	NCR	0.820 ± 0.043	0.545 ± 0.142	0.885 ± 0.034	0.818 ± 0.044	0.515 ± 0.010
	TLL	0.760 ± 0.044	0.543 ± 0.149	0.833 ± 0.039	0.765 ± 0.044	0.511 ± 0.011
	RUS	0.868 ± 0.033	0.657 ± 0.151	0.879 ± 0.037	0.880 ± 0.034	0.709 ± 0.043
	RBU	0.902 ± 0.035	0.835 ± 0.110	0.917 ± 0.031	0.915 ± 0.034	0.854 ± 0.087
wdbc	ENN	0.969 ± 0.015	0.894 ± 0.059	0.956 ± 0.023	0.970 ± 0.016	0.963 ± 0.018
	AKNN	0.967 ± 0.015	0.893 ± 0.057	0.954 ± 0.017	0.970 ± 0.015	0.961 ± 0.016
	NM1	0.950 ± 0.020	0.838 ± 0.077	0.905 ± 0.024	0.951 ± 0.023	0.935 ± 0.024
	NM2	0.944 ± 0.022	0.817 ± 0.097	0.909 ± 0.023	0.948 ± 0.020	0.934 ± 0.027
	NM3	0.892 ± 0.054	0.680 ± 0.156	0.805 ± 0.060	0.907 ± 0.053	0.879 ± 0.056
	NUS1	0.960 ± 0.023	0.974 ± 0.018	0.975 ± 0.015	0.964 ± 0.021	0.978 ± 0.014
	NUS2	0.959 ± 0.019	0.970 ± 0.017	0.976 ± 0.015	0.965 ± 0.016	0.977 ± 0.014
	CC	0.948 ± 0.020	0.851 ± 0.064	0.905 ± 0.030	0.950 ± 0.023	0.935 ± 0.023
	NCR	0.969 ± 0.017	0.885 ± 0.068	0.951 ± 0.021	0.974 ± 0.014	0.966 ± 0.018
	TLL	0.956 ± 0.020	0.864 ± 0.076	0.938 ± 0.020	0.957 ± 0.017	0.948 ± 0.020
	RUS	0.948 ± 0.019	0.848 ± 0.071	0.925 ± 0.028	0.964 ± 0.016	0.945 ± 0.021
	RBU	0.948 ± 0.048	0.830 ± 0.188	0.920 ± 0.050	0.957 ± 0.042	0.940 ± 0.042
yeast_4	ENN	0.711 ± 0.070	0.669 ± 0.112	0.665 ± 0.074	0.642 ± 0.067	0.625 ± 0.070
	AKNN	0.722 ± 0.078	0.681 ± 0.123	0.668 ± 0.066	0.660 ± 0.057	0.632 ± 0.072
	NM1	0.661 ± 0.080	0.636 ± 0.092	0.647 ± 0.092	0.697 ± 0.089	0.684 ± 0.085
	NM2	0.864 ± 0.078	0.583 ± 0.140	0.718 ± 0.093	0.875 ± 0.074	0.661 ± 0.093
	NM3	0.625 ± 0.089	0.584 ± 0.122	0.662 ± 0.104	0.615 ± 0.107	0.678 ± 0.098
	NUS1	0.950 ± 0.059	0.951 ± 0.046	0.955 ± 0.042	0.928 ± 0.063	0.945 ± 0.045
	NUS2	0.805 ± 0.062	0.736 ± 0.135	0.818 ± 0.066	0.753 ± 0.066	0.774 ± 0.075
	CC	0.893 ± 0.075	0.746 ± 0.115	0.778 ± 0.084	0.889 ± 0.072	0.789 ± 0.078
	NCR	0.684 ± 0.072	0.634 ± 0.121	0.658 ± 0.064	0.613 ± 0.057	0.606 ± 0.061
	TLL	0.649 ± 0.062	0.630 ± 0.111	0.601 ± 0.066	0.581 ± 0.055	0.579 ± 0.053
	RUS	0.818 ± 0.075	0.783 ± 0.096	0.824 ± 0.086	0.852 ± 0.077	0.825 ± 0.085
	RBU	0.851 ± 0.153	0.772 ± 0.179	0.819 ± 0.159	0.861 ± 0.153	0.830 ± 0.129
yeast_5	ENN	0.931 ± 0.025	0.875 ± 0.049	0.886 ± 0.028	0.912 ± 0.030	0.889 ± 0.026
	AKNN	0.935 ± 0.027	0.881 ± 0.036	0.890 ± 0.034	0.916 ± 0.031	0.892 ± 0.031
	NM1	0.815 ± 0.034	0.779 ± 0.069	0.810 ± 0.038	0.814 ± 0.037	0.830 ± 0.041
	NM2	0.898 ± 0.031	0.839 ± 0.047	0.893 ± 0.029	0.904 ± 0.027	0.890 ± 0.024
	NM3	0.768 ± 0.058	0.702 ± 0.069	0.717 ± 0.049	0.763 ± 0.049	0.786 ± 0.047
	NUS1	0.943 ± 0.028	0.910 ± 0.037	0.964 ± 0.022	0.948 ± 0.027	0.935 ± 0.027
	NUS2	0.944 ± 0.024	0.910 ± 0.049	0.964 ± 0.019	0.948 ± 0.024	0.938 ± 0.028
	CC	0.933 ± 0.030	0.864 ± 0.035	0.901 ± 0.035	0.941 ± 0.025	0.907 ± 0.035
	NCR	0.930 ± 0.032	0.871 ± 0.043	0.896 ± 0.031	0.911 ± 0.033	0.885 ± 0.033
	TLL	0.880 ± 0.037	0.832 ± 0.060	0.852 ± 0.038	0.858 ± 0.042	0.849 ± 0.035
	RUS	0.925 ± 0.030	0.863 ± 0.039	0.888 ± 0.040	0.920 ± 0.031	0.891 ± 0.031
	RBU	0.922 ± 0.065	0.875 ± 0.078	0.896 ± 0.074	0.920 ± 0.059	0.896 ± 0.069
yeast_6	ENN	0.764 ± 0.079	0.718 ± 0.114	0.810 ± 0.077	0.734 ± 0.077	0.692 ± 0.072
	AKNN	0.789 ± 0.085	0.762 ± 0.107	0.826 ± 0.090	0.753 ± 0.087	0.727 ± 0.082
	NM1	0.649 ± 0.109	0.587 ± 0.102	0.707 ± 0.096	0.694 ± 0.116	0.661 ± 0.119
	NM2	0.837 ± 0.082	0.673 ± 0.121	0.871 ± 0.080	0.817 ± 0.096	0.743 ± 0.118
	NM3	0.684 ± 0.112	0.616 ± 0.106	0.701 ± 0.113	0.716 ± 0.109	0.651 ± 0.130
	NUS1	0.913 ± 0.070	0.967 ± 0.043	0.911 ± 0.071	0.920 ± 0.068	0.969 ± 0.048
	NUS2	0.830 ± 0.094	0.783 ± 0.124	0.914 ± 0.059	0.799 ± 0.098	0.787 ± 0.097
	CC	0.914 ± 0.083	0.777 ± 0.111	0.820 ± 0.100	0.906 ± 0.093	0.810 ± 0.096
	NCR	0.772 ± 0.090	0.677 ± 0.143	0.822 ± 0.082	0.734 ± 0.073	0.665 ± 0.078
	TLL	0.708 ± 0.082	0.653 ± 0.131	0.771 ± 0.076	0.683 ± 0.083	0.636 ± 0.085
	RUS	0.813 ± 0.099	0.803 ± 0.107	0.894 ± 0.070	0.870 ± 0.070	0.874 ± 0.074
	RBU	0.833 ± 0.195	0.804 ± 0.221	0.854 ± 0.164	0.879 ± 0.157	0.823 ± 0.216

(supplementary file)]. We do note that, achieving complete balance, through removing many majority samples (as is done by NUS-1 and many other approaches) may be problematic in some real-life instances. In such scenarios, NUS-2 may be a better alternative.

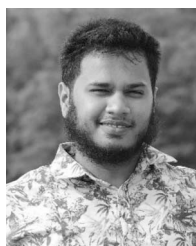
Both NUS-1 and NUS-2 attempt to ensure better separability of the remaining data points (i.e., after removal of selected majority samples). This is particularly useful in the medical domain where the minority points (points of interest)

are overlapped with majority points making the classification extremely hard.

REFERENCES

- [1] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, Sep. 2009.
- [2] D. A. Cieslak, N. V. Chawla, and A. Striegel, "Combating imbalance in network intrusion datasets," in *Proc. IEEE Int. Conf. Granular Comput. (GRC)*, Atlanta, GA, USA, 2006, pp. 732–737.

- [3] W. Wei, J. Li, L. Cao, Y. Ou, and J. Chen, "Effective detection of sophisticated online banking fraud on extremely imbalanced data," *World Wide Web*, vol. 16, no. 4, pp. 449–475, 2013.
- [4] T. Fawcett and F. Provost, "Adaptive fraud detection," *Data Min. Knowl. Discover.*, vol. 1, no. 3, pp. 291–316, 1997.
- [5] J. Burez and D. Van den Poel, "Handling class imbalance in customer churn prediction," *Expert Syst. Appl.*, vol. 36, no. 3, pp. 4626–4636, 2009.
- [6] Y. Liu, H. T. Loh, and A. Sun, "Imbalanced text classification: A term weighting approach," *Expert Syst. Appl.*, vol. 36, no. 1, pp. 690–701, 2009.
- [7] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "Smote: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, Jun. 2002.
- [8] H. He, Y. Bai, E. A. Garcia, and S. Li, "ADASYN: Adaptive synthetic sampling approach for imbalanced learning," in *Proc. IEEE Int. Joint Conf. Neural Netw. (IEEE World Congr. Comput. Intell.)*, Hong Kong, China, 2008, pp. 1322–1328.
- [9] S. Barua, M. M. Islam, X. Yao, and K. Murase, "MWMOTE—majority weighted minority oversampling technique for imbalanced data set learning," *IEEE Trans. Knowl. Data Eng.*, vol. 26, no. 2, pp. 405–425, Feb. 2014.
- [10] H. Han, W.-Y. Wang, and B.-H. Mao, "Borderline-SMOTE: A new over-sampling method in imbalanced data sets learning," in *Proc. Int. Conf. Intell. Comput.*, 2005, pp. 878–887.
- [11] G. E. Batista, R. C. Prati, and M. C. Monard, "A study of the behavior of several methods for balancing machine learning training data," *ACM SIGKDD Explorations Newsletter*, vol. 6, no. 1, pp. 20–29, 2004.
- [12] D. L. Wilson, "Asymptotic properties of nearest neighbor rules using edited data," *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-2, no. 3, pp. 408–421, Jul. 1972.
- [13] J. Laurikkala, "Improving identification of difficult small classes by balancing class distribution," in *Proc. Conf. Artif. Intell. Med. Eur.*, 2001, pp. 63–66.
- [14] I. Tomek, "A generalization of the k-NN rule," *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-6, no. 2, pp. 121–126, Feb. 1976.
- [15] L. Feng, H. Wang, B. Jin, H. Li, M. Xue, and L. Wang, "Learning a distance metric by balancing KL-divergence for imbalanced datasets," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 49, no. 12, pp. 2384–2395, Dec. 2019.
- [16] X. Gu, F. Chung, H. Ishibuchi, and S. Wang, "Imbalanced TSK fuzzy classifier by cross-class Bayesian fuzzy clustering and imbalance learning," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 47, no. 8, pp. 2005–2020, Aug. 2017.
- [17] T. Khoshgoftaar, J. Van Hulse, and A. Napolitano, "Comparing boosting and bagging techniques with noisy and imbalanced data," *IEEE Trans. Syst., Man, Cybern. A, Syst. Humans*, vol. 41, no. 3, pp. 552–568, May 2011.
- [18] C. Seiffert, T. M. Khoshgoftaar, and J. Van Hulse, "Improving software-quality predictions with data sampling and boosting," *IEEE Trans. Syst., Man, Cybern. A, Syst. Humans*, vol. 39, no. 6, pp. 1283–1294, Nov. 2009.
- [19] C. Seiffert, T. M. Khoshgoftaar, J. Van Hulse, and A. Napolitano, "RUSBoost: A hybrid approach to alleviating class imbalance," *IEEE Trans. Syst., Man, Cybern. A, Syst. Humans*, vol. 40, no. 1, pp. 185–197, Jan. 2010.
- [20] T. Jo and N. Japkowicz, "Class imbalances versus small disjuncts," *ACM SIGKDD Explorations Newsletter*, vol. 6, no. 1, pp. 40–49, 2004.
- [21] D. Mease, A. J. Wyner, and A. Buja, "Boosted classification trees and class probability/quantile estimation," *J. Mach. Learn. Res.*, vol. 8, pp. 409–439, Mar. 2007.
- [22] K. Jiang, J. Lu, and K. Xia, "A novel algorithm for imbalance data classification based on genetic algorithm improved smote," *Arab. J. Sci. Eng.*, vol. 41, no. 8, pp. 3255–3266, 2016.
- [23] I. Mani and I. Zhang, "KNN approach to unbalanced data distributions: A case study involving information extraction," in *Proc. Workshop Learn. Imbalanced Datasets*, vol. 126, 2003, pp. 42–48.
- [24] G. Lemaître, F. Nogueira, and C. K. Aridas, "Imbalanced-learn: A python toolbox to tackle the curse of imbalanced datasets in machine learning," *J. Mach. Learn. Res.*, vol. 18, no. 17, pp. 1–5, 2017.
- [25] M. Koziarski, "Radial-based undersampling algorithm for classification of breast cancer histopathological images affected by data imbalance," in *Proc. 12th Int. Congr. Image Signal Process. BioMed. Eng. Inform. (CISP-BMEI)*, Suzhou, China, 2019, pp. 1–5.
- [26] S. García and F. Herrera, "Evolutionary undersampling for classification with imbalanced datasets: Proposals and taxonomy," *Evol. Comput.*, vol. 17, no. 3, pp. 275–306, Sep. 2009.
- [27] Y. Hou, B. Li, L. Li, and J. Liu, "A density-based under-sampling algorithm for imbalance classification," *J. Phys. Conf. Series*, vol. 1302, no. 2, Aug. 2019, Art. no. 022064.
- [28] Y. Liu, Y. Wang, X. Ren, H. Zhou, and X. Diao, "A classification method based on feature selection for imbalanced data," *IEEE Access*, vol. 7, pp. 81794–81807, 2019.
- [29] X. Chen, Q. Kang, M. Zhou, and Z. Wei, "A novel under-sampling algorithm based on iterative-partitioning filters for imbalanced classification," in *Proc. IEEE Int. Conf. Autom. Sci. Eng. (CASE)*, Fort Worth, TX, USA, 2016, pp. 490–494.
- [30] W. W. Ng, J. Hu, D. S. Yeung, S. Yin, and F. Roli, "Diversified sensitivity-based undersampling for imbalance classification problems," *IEEE Trans. Cybern.*, vol. 45, no. 11, pp. 2402–2412, Nov. 2015.
- [31] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, Oct. 2011.
- [32] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Stat.*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [33] T. Zhang, "Solving large scale linear prediction problems using stochastic gradient descent algorithms," in *Proc. 21st Int. Conf. Mach. Learn.*, 2004, p. 116.
- [34] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.
- [35] R. Duda and P. Hart, *Pattern Classification and Scene Analysis*. New York, NY, USA: Wiley, 1973.
- [36] D. G. Kleinbaum and M. Klein, *Logistic Regression*. New York, NY, USA: Springer, 2002.
- [37] S. Van Der Walt, S. C. Colbert, and G. Varoquaux, "The NumPy array: A structure for efficient numerical computation," *Comput. Sci. Eng.*, vol. 13, no. 2, pp. 22–30, Mar./Apr. 2011.
- [38] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Comput. Sci. Eng.*, vol. 9, no. 3, pp. 90–95, May/Jun. 2007.
- [39] F. Chollet *et al.* (2015). *Keras*. [Online]. Available: <https://keras.io>
- [40] D. Dua and C. Graff, "UCI machine learning repository," 2017.
- [41] J. Alcalá-Fdez *et al.*, "KEEL data-mining software tool: Data set repository, integration of algorithms and experimental analysis framework," *J. Multiple Valued Logic Soft Comput.*, vol. 17, nos. 2–3, pp. 255–287, 2011.
- [42] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognit. Lett.*, vol. 27, no. 8, pp. 861–874, 2006.
- [43] F. Wilcoxon, "Individual comparisons by ranking methods," in *Breakthroughs in Statistics*. New York, NY, USA: Springer, 1992, pp. 196–202.



Md Adnan Arefeen received the Bachelor of Science degree in computer science and engineering from the Bangladesh University of Engineering and Technology, Dhaka, Bangladesh, in 2017. He is currently pursuing the iPHD degree with the Department of Computer Science Electrical Engineering, University of Missouri–Kansas City, Kansas City, MO, USA.

He is currently on study leave from United International University (UIU), Dhaka, where he was working as a Lecturer with the Department of Computer Science and Engineering. He worked as a Software Engineer with iOS Department, REVE Systems Ltd., Singapore. He was also acting as a moderator of UIU APP Forum with UIU. His other research areas include but not limited to edge intelligence, machine learning, data mining, neural network, and meta-heuristics.



Sumaiya Tabassum Nimi received the Bachelor of Science degree in computer science and engineering from the Bangladesh University of Engineering and Technology, Dhaka, Bangladesh, in 2017. She is currently pursuing the Ph.D. degree with the Department of CSEE, University of Missouri–Kansas City, Kansas City, MO, USA.

She is on study leave from United International University, Dhaka, where she served as a Lecturer. Formerly, she worked as Lecturer with Eastern University, Dhaka. Her research area includes deep

learning, data mining, meta-heuristics.



M. Sohel Rahman (Senior Member, IEEE) received the B.Sc. and M.Sc. degrees in computer science and engineering from the Bangladesh University of Engineering and Technology (BUET), Dhaka, Bangladesh, in 2002 and 2004, respectively, and the Ph.D. degree in computer science from King's College London, London, U.K., in 2008.

He is currently a Professor with the Department of Computer Science and Engineering, BUET. It is a premiere university in the field of Engineering and Technology in Bangladesh. He had worked as a

Visiting Research Fellow of King's College London, from 2008 to 2011, where he was a Visiting Senior Research Fellow from 2014 to 2015. He has so far published 80 peer-reviewed international journal papers. He has led research and development projects funded by the British Council, UGC-World Bank, and BUET. Among his highly cited results are the work on high dimensional Knapsack problems, algorithms on different variants of sequence alignment problems, data structures for different variants of string and sequence matching and sufficient conditions for Hamiltonicity and metaheuristics solutions for hard real-life problems in different branches of science and engineering. His research interest includes, but is not limited to, theory and algorithms, stringology, metaheuristics, applied informatics, computational biology and bioinformatics.

Prof. Rahman received different scholarships and fellowships, including Commonwealth Scholarship, Commonwealth Fellowship, ACU Titular Fellowship, University College London-Big Data Institute visiting grant, and London Mathematical Society Visiting Grant. He is also a recipient of the Bangladesh Academy of Sciences Gold Medal and UGC Award. He is a member of the American Mathematical Society and London Mathematical Society. He is also a Peer-review Associate College Member of EPSRC, U.K. He is currently serving as an Academic Editor for *PLoS ONE* and an Associate Editor for *BMC Research Notes*. He has served/are serving as a Guest Editor for the *Journal of Graph Algorithms and Applications*, the *Journal of Discrete Algorithms*, *Fundamenta Informaticae*, and *Theoretical Computer Science*. He has also served as Program Committee members in a number of conference series' of international repute. He has collaborated with eminent scientists and researchers from both home and abroad. He regularly writes reviews at Mathematical Review and Computing Review. He is a Senior Member of ACM.